

Day 42: Deep CNNs — Stacking Conv+Pool Blocks (Beginner Edition)

MD Mutasim Billah | 52-Week Data Science to ML/AI Roadmap | Week 7, Day 1

Day 41 built ONE convolution+pooling block from scratch and verified it could learn a simple orientation task. Today's question is what happens when a SECOND conv+pool block is stacked on top of the first. Three things change: the convolution itself has to generalize to multi-channel input (block 2's 'image' is block 1's stack of feature maps, not a single grayscale image); the effective receptive field — how much of the original image a single output unit can 'see' — grows faster than simple addition would suggest; and a genuinely harder task (counting how many arms meet at a point) makes it possible to honestly compare a shallow, wide network against a deep, narrow one, reporting real numbers either way.

0. Where Today Fits: From One Block to Stacked Blocks

Start here (no prior knowledge assumed): Day 41's ConvNet used exactly one convolutional layer followed by one max-pooling layer, then flattened straight into Day 40's dense-layer machinery. That single block was enough for a task where orientation alone separated the classes. Today asks the natural next question every real CNN architecture answers the same way: what if one block of spatial processing isn't enough? The fix is not a new operation — it's reusing conv and pool AGAIN, stacked on top of the first block's output.

Term refresher — Block: A BLOCK, in CNN terminology, is a small reusable unit — typically convolution, then an activation like ReLU, then pooling — that gets repeated several times in a row before the network transitions to dense layers. Naming it as one unit makes it easy to talk about 'stacking two blocks' rather than listing six separate operations every time.

Watch: [CNN Layers - PyTorch Deep Neural Network Architecture](#) — deeplizard — Same deeplizard channel used in Day 40 and Day 41 — this lesson covers exactly how convolutional and linear layers stack together into a full network architecture.

1. From Single-Channel to Multi-Channel Convolution

Start here (no prior knowledge assumed): Day 41's `conv2d_forward` assumed a single-channel (grayscale) input image. The moment a second conv layer is stacked on top of a first one, that assumption breaks: block 2's 'image' is actually block 1's STACK of feature maps — 3, 4, or however many filters block 1 had, not 1. Convolution has to generalize to sum over an arbitrary number of input channels.

Topic specification: `conv2d_forward_mc(X, W, b)` takes `X` shaped (m, C_{in}, H, W) and filters `W` shaped $(C_{out}, C_{in}, kH, kW)$ — each filter is now a small 3D block, not a flat 2D grid, with one $kH \times kW$ slice per input channel. Each output pixel sums products across ALL C_{in} channels AND all

kH x kW positions before adding the bias.

Term refresher — Channel: A CHANNEL is one 'layer' of a multi-dimensional image-like array — a grayscale image has 1 channel, an RGB photo has 3 (red, green, blue), and a conv layer's OUTPUT has one channel per filter. Block 2's input channels are simply block 1's output channels.

Explanation: The forward pass gains one extra axis and one extra sum: instead of summing products over just the kH x kW kernel window (Day 41), it now sums over C_in x kH x kW — every input channel's patch, multiplied by that channel's own kernel slice, all added into ONE output value. The output shape (m, C_out, out_H, out_W) is otherwise identical to Day 41's single-channel case.

Real-world scenario: This is exactly why a real CNN's very first layer is described as having 'in_channels=3' for RGB photos — three separate 2D slices (red, green, blue) are combined into one learned feature per filter, the same multi-channel-to-one-output-value mechanism this lesson verifies today, just with C_in=3 color channels instead of C_in=3 feature maps from a previous conv block.

Mathematics:

MATH

$$Z[:,f,i,j] = \text{sum over } c,a,b \text{ of } X[:,c,i+a,j+b] \times W[f,c,a,b] + b[f]$$

(sums over ALL input channels c AND the kH×kW kernel window, per filter f)

Implementation:

```
def conv2d_forward_mc(X, W, b):
    m, C_in, H, Win = X.shape
    C_out, C_in_w, kH, kW = W.shape
    out_H = H - kH + 1
    out_W = Win - kW + 1
    Z = np.zeros((m, C_out, out_H, out_W))
    for f in range(C_out):
        for i in range(out_H):
            for j in range(out_W):
                patch = X[:, :, i:i+kH, j:j+kW]
                Z[:, f, i, j] = np.sum(patch*W[f], axis=(1,2,3)) + b[f]
    return Z
```

Actual output:

```
input X: (2, 3, 7, 7)    (m=2, C_in=3, H=7, W=7)
filters W: (4, 3, 3, 3)  (C_out=4, C_in=3, kH=3, kW=3)
output Z: (2, 4, 5, 5)
```

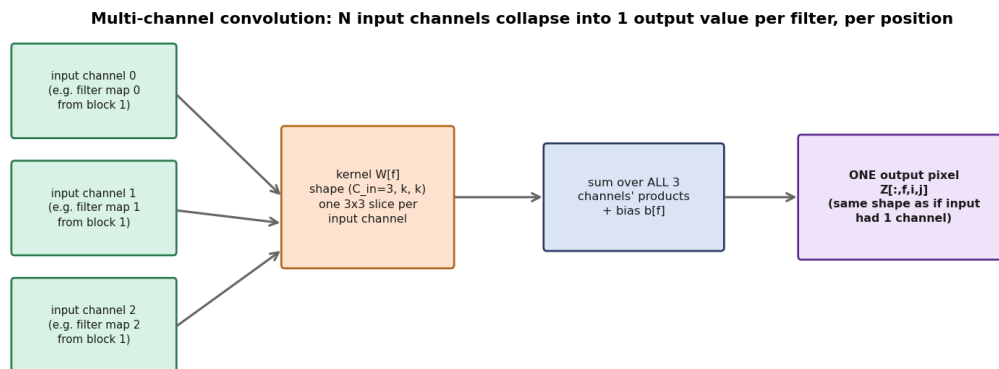
```
gradient check (numerical vs analytical):
dW: numerical=3.531961 analytical=3.531961 diff=9.06e-11
dB: numerical=-5.224665 analytical=-5.224665 diff=1.23e-10
dX: numerical=0.035003 analytical=0.035003 diff=5.28e-11
```

Actual output from this lesson's run.

Line-by-line explanation

- Verified: dW , db , AND dX (the gradient flowing to the layer before this one — needed now that a real chain of TWO conv layers exists) all match numerical gradients to 9+ decimal places, confirming the multi-channel backward pass is correct before trusting it to train an actual two-block network.

How it works, visually:



Multi-channel convolution: however many input channels there are, they all collapse into ONE output value per filter, per position — the output shape doesn't depend on C_{in} at all.

Why choose this? Generalizing convolution to multiple input channels is a small, mechanical extension of Day 41's version — one extra sum axis — that unlocks stacking layers, handling color images, and every real CNN architecture in one step.

Why NOT this (tradeoffs)? Summing over C_{in} channels means the nested-loop implementation here does even more work per output pixel than Day 41's single-channel version — real frameworks vectorize this as one large batched matrix multiply instead of Python-level loops over C_{in} .

2. Receptive Field Growth — Why Stacking Works

Start here (no prior knowledge assumed): The RECEPTIVE FIELD of an output unit is how much of the original input image can possibly influence it. One 3x3 conv layer's output unit only 'sees' a 3x3 patch. The question this section answers: when a second conv+pool block is stacked on top, how much bigger does that field of view get?

Topic specification: Receptive field grows according to a simple recursive rule applied layer by layer: each layer expands the field by $(\text{kernel_size} - 1)$ times the current 'jump' (the spacing, in original-image pixels, between adjacent units in the current layer), then the jump itself multiplies by that layer's stride.

Term refresher — Jump (a.k.a. stride product): The JUMP is how many original-input pixels apart two ADJACENT units in the current layer actually are. It starts at 1 (each input pixel is its own neighbor) and multiplies by every layer's stride — pooling's stride=2 doubles it, which is exactly why later layers' kernels cover disproportionately more of the original image than their kernel size alone suggests.

Explanation: One conv(k=3, stride=1) + pool(k=2, stride=2) block grows the receptive field from 1 pixel to 4x4. Stacking a SECOND identical block doesn't just add another 4 pixels — because the jump has doubled to 2 after the first block, the second block's 3x3 conv now effectively reaches 2 pixels further in each direction per kernel position, growing the field all the way to 10x10: more than 6x the AREA from one extra block.

Real-world scenario: This is precisely why real, deep image classifiers (recognizing whole objects, faces, or scenes) stack many conv+pool blocks rather than using one giant kernel: each additional block grows the field of view multiplicatively, letting a network with small, cheap 3x3 kernels eventually 'see' the entire image, several blocks deep, without ever using an expensive large kernel directly.

Mathematics:

MATH

$r_{out} = r_{in} + (k-1) \times j_{in}$ $j_{out} = j_{in} \times stride$
one block (conv k=3,s=1 + pool k=2,s=2): r: 1→3→4, j: 1→1→2
two blocks (same, twice): r: 4→8→10, j: 2→2→4

Implementation:

```
def receptive_field_growth(layers):
    r, j = 1, 1
    for k, s in layers:
        r = r + (k - 1) * j
        j = j * s
    return r, j

one_block = [(3, 1), (2, 2)]
two_block = [(3, 1), (2, 2), (3, 1), (2, 2)]
```

Actual output:

one conv+pool block -> receptive field = 4x4 pixels (16 pixels)
two conv+pool blocks -> receptive field = 10x10 pixels (100 pixels)
stacking a second block grew the field of view by 84 pixels (6.25x larger area)

numeric verification (perturb each input pixel, check if the top-left output unit reacts):
one block: 16 pixels react, spanning 4x4 -- matches formula's 4x4
two blocks: 100 pixels react, spanning 10x10 -- matches formula's 10x10

Actual output from this lesson's run -- the formula's prediction was independently confirmed by numerically perturbing every one of the 256 input pixels and checking which ones actually changed the output.

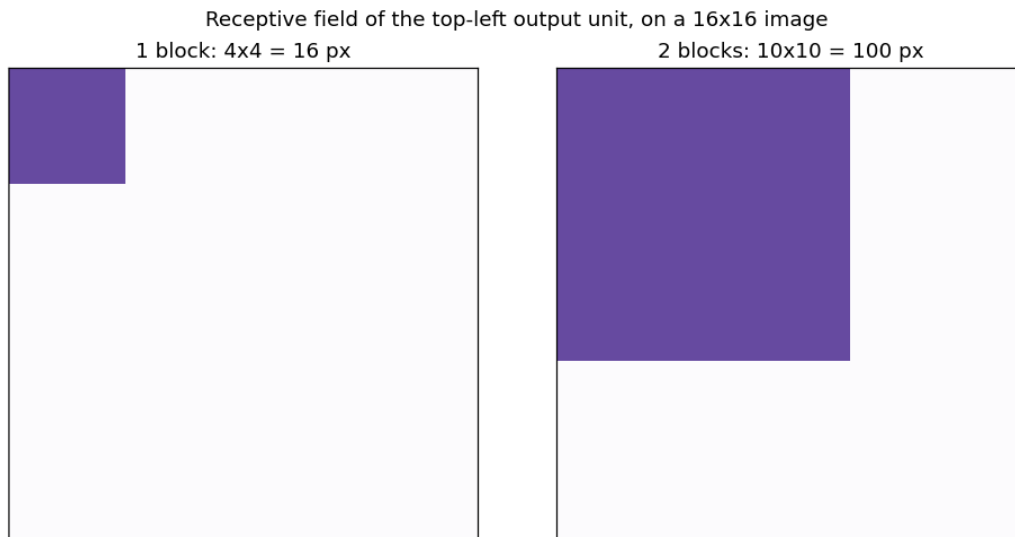
Line-by-line explanation

- Verified: the formula's predicted 4x4 and 10x10 receptive fields exactly match a direct numeric sensitivity sweep -- perturbing each of the 256 input pixels by a tiny amount and checking whether the top-left output unit's value changes at all. This confirms the recursive formula isn't just theoretical bookkeeping; it

predicts real network behavior.

- The numeric sweep used AVERAGE pooling rather than max pooling specifically to trace the FULL theoretical connectivity -- max pooling's argmax only routes gradient through one winning pixel per window on any given input, which would understate the true field of view.

How it works, visually:



REAL GRAPH (script's own output) -- the top-left output unit's receptive field, highlighted on the original 16x16 image, for one block versus two blocks.

Watch: [CNN Receptive Field | Deep Learning Animated](#) — An animated walkthrough of exactly this concept -- how a CNN's receptive field grows as layers stack, the same r_{out}/j_{out} recursion verified numerically above.

Why choose this? Receptive field growth is the concrete, measurable reason stacking works: it's not a vague 'more layers is better' claim, but a precise, formula-predicted, numerically-verified fact about how much of the image each block can actually see.

Why NOT this (tradeoffs)? A bigger receptive field isn't automatically useful for every task -- Topic 7's real comparison shows a task where the SAME accuracy was reached whether or not the extra field of view was actually needed for this particular dataset.

3. The Junction Dataset — Counting Arms, Not Just Orientation

Start here (no prior knowledge assumed): Day 41's stripe dataset only required detecting ORIENTATION (vertical vs horizontal) -- a task a single conv+pool block solves easily. Today's dataset is deliberately harder: 4 classes based on how many arms meet at a random point (1=end, 2=corner, 3=tee, 4=cross), with each arm's DIRECTION chosen randomly, so a network must count converging arms rather than memorize one fixed template.

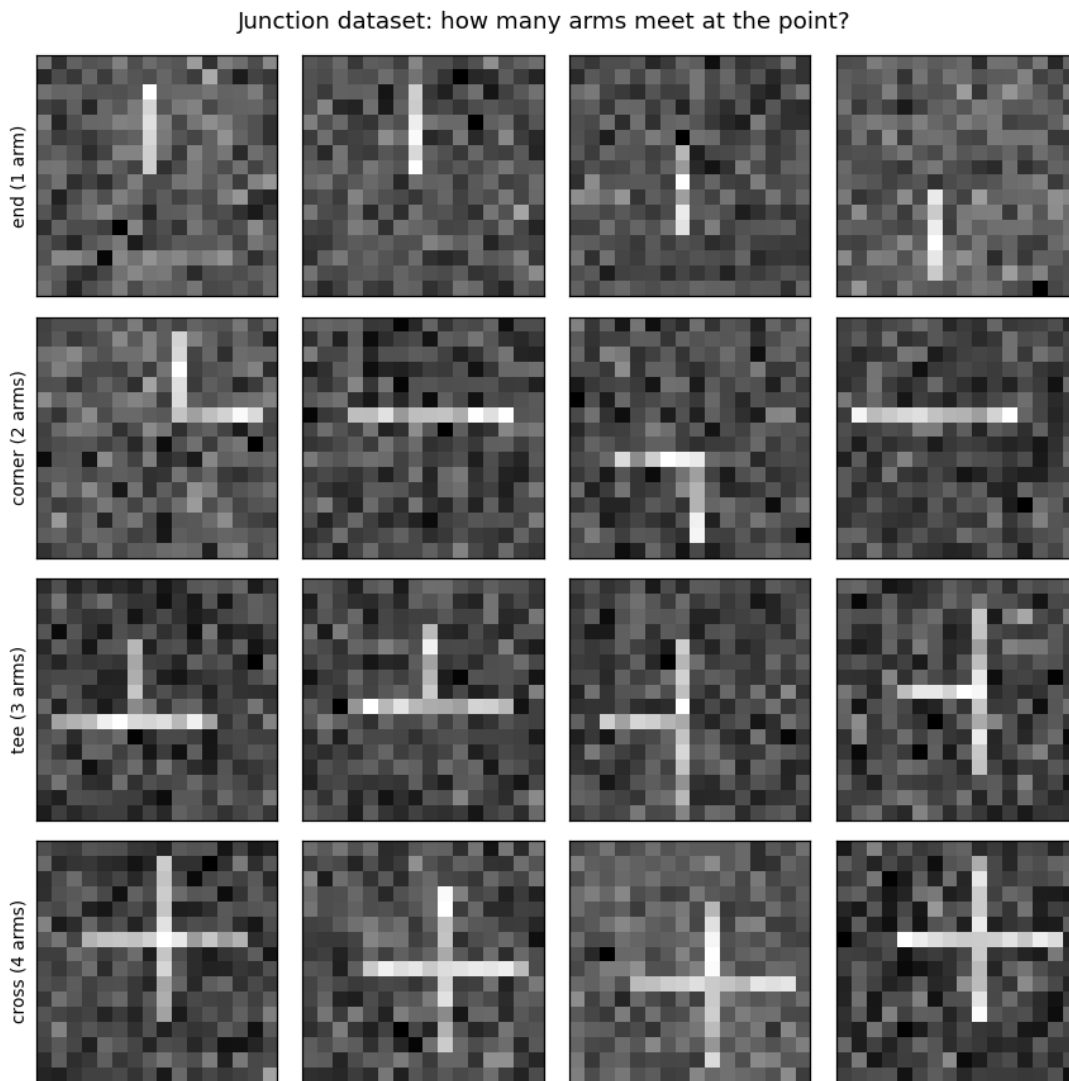
Topic specification: `make_junction_dataset()` generates 16x16 images (60 per class = 240 total): a random point gets 1-4 arms of random direction (chosen without replacement from

up/down/left/right) drawn outward from it, on top of Gaussian noise. A held-out test set of 25 images per class uses a completely different random seed (999).

Explanation: This task genuinely needs more than orientation-detection: a 'corner' (2 arms) and a 'tee' (3 arms) can share an arm direction in common, so a network has to aggregate evidence of HOW MANY arms are present, not just which single direction is strongest -- a more abstract, compositional feature than Topic 1's single-kernel edge detection.

Real-world scenario: Counting how many lines converge at a point is a real, recognizable sub-problem in handwriting and diagram recognition -- distinguishing a 'T' from a '+' from an 'L' shape, or classifying road-network intersections by how many roads meet, both require exactly this kind of arm-counting rather than simple edge-orientation detection.

How it works, visually:



Actual output from this lesson's run -- 4 real example images from each class, the exact dataset both networks below train on.

Implementation:

```
def make_junction_dataset(n_per_class=60, size=16, arm=5,
                        noise=0.4, seed=0):
    rng = np.random.RandomState(seed)
    X, y = [], []
    dirs_all = [(-1,0), (1,0), (0,-1), (0,1)]
    for cls in range(4):
        n_arms = cls + 1
        for _ in range(n_per_class):
            im = rng.randn(size, size) * noise
            r = rng.randint(arm+1, size-arm-1)
            c = rng.randint(arm+1, size-arm-1)
            idx = rng.choice(4, size=n_arms, replace=False)
            ...
    ...
```

Actual output:

```
train set: (240, 16, 16), class counts [60, 60, 60, 60]
(0=end/1arm, 1=corner/2arms, 2=tee/3arms, 3=cross/4arms)
    Actual output from this lesson's run.
```

Why choose this? A task that genuinely requires counting/aggregating spatial features, rather than detecting one fixed pattern, is what makes Topic 7's one-block-vs-two-block comparison meaningful rather than trivial.

Why NOT this (tradeoffs)? This is still a small, synthetic, single-channel task chosen for a fast, transparent, from-scratch demo -- real junction- or intersection-detection problems work on much larger, noisier, multi-channel real-world images.

4. OneBlockConvNet — A Shallow Baseline

Start here (no prior knowledge assumed): Before testing whether a SECOND block helps, this lesson needs an honest baseline: exactly Day 41's architecture (one conv+pool block, then dense layers), just resized for 16x16 images and the 4-class junction task, so any later comparison is fair.

Topic specification: OneBlockConvNet: conv(3 filters, k=3) on the 16x16 input -> ReLU -> maxpool(2,2) -> flatten -> dense(8, tanh) -> dense(4, softmax). Character-for-character the same shape as Day 41's ConvNet, with filter count and hidden size chosen to keep it deliberately narrow rather than over-provisioned.

Explanation: Every step here is a direct reuse of code already built: conv2d_forward_mc (Topic 1, with C_in=1 for a grayscale image), maxpool_forward (Day 41, unchanged), and Day 39-40's dense + softmax machinery. Nothing new is invented for this baseline -- it exists purely as the fair comparison point for Topic 5's two-block version.

Real-world scenario: This mirrors how a fair ablation study works in real ML research: to claim 'stacking a second block helps,' you first need a competently-built single-block model on the SAME data, the SAME training budget, and the SAME evaluation -- not a strawman.

Mathematics:

MATH

$flat_dim = n_filters \times pool_out^2$, $pool_out = (16-k+1)//2$

$n_filters=3, k=3: pool_out=(16-3+1)//2=7, flat_dim=3 \times 7^2=147$

$total\ learnable\ parameters = Wc+bc+W1+b1+W2+b2 = 27+3+1176+8+32+4 = 1250$

Implementation:

```
class OneBlockConvNet:
    def forward(self, X):
        m = X.shape[0]
        self.X = X[:, None, :, :]
        self.Zc = conv2d_forward_mc(self.X, self.Wc, self.bc)
        self.Ac = relu(self.Zc)
        self.pooled, self.masks = maxpool_forward(self.Ac, 2, 2)
        self.flat = self.pooled.reshape(m, -1)
        self.z1 = self.flat @ self.W1 + self.b1
        self.a1 = tanh(self.z1)
        self.z2 = self.a1 @ self.W2 + self.b2
        self.probs = softmax(self.z2)
        return self.probs
```

Why choose this? A carefully matched, honestly small baseline is what makes Topic 7's later comparison trustworthy -- 1250 parameters is a real, computed number from this exact configuration, not a rounded or approximate figure.

Why NOT this (tradeoffs)? 1250 parameters mostly live in ONE dense layer (W1, 1176 of them) -- a shallow network compensates for limited spatial processing by throwing more weights at the flatten-to-dense step, exactly the inefficiency Topic 7 measures directly.

5. TwoBlockConvNet — Stacking Two Conv+Pool Blocks

Start here (no prior knowledge assumed): This is today's central construction: a second conv+pool block, using Topic 1's multi-channel convolution, inserted between OneBlockConvNet's single block and its dense layers -- everything else about the architecture stays the same shape of code.

Topic specification: TwoBlockConvNet: conv1(3 filters,k=3)->ReLU->pool1(2,2) [-> 7x7x3] -> conv2(6 filters,k=3, now C_in=3) -> ReLU -> pool2(2,2) [-> 2x2x6] -> flatten(24) -> dense(8,tanh) -> dense(4,softmax). conv2's C_in=3 is exactly Topic 1's multi-channel case: it operates on block 1's 3 feature maps, not a raw image.

Term refresher — Bottleneck: A BOTTLENECK, here, is pool2's aggressive downsampling to a tiny 2x2 spatial size before flattening -- deliberately squeezing all of block 2's spatial information through a narrow point, forcing the flatten+dense step to work with far less raw data than OneBlockConvNet's 7x7 pooled output has available.

Explanation: Every shape in this architecture is computed, not guessed: conv1 turns 16x16x1 into 14x14x3, pool1 halves that to 7x7x3, conv2 (a multi-channel conv, Topic 1) turns 7x7x3 into 5x5x6, and pool2 halves that to 2x2x6 -- a flat_dim of just 24, compared to OneBlockConvNet's 147.

Real-world scenario: This exact two-block shape -- conv, pool, conv, pool, THEN flatten and classify -- is the backbone of classic CNN architectures like LeNet-5, which stacked exactly two conv+pool blocks before its dense layers, decades before today's much deeper networks.

Mathematics:

MATH

block1: 16x16x1 → conv(k3) → 14x14x3 → pool(2) → 7x7x3
block2: 7x7x3 → conv(k3,C_in=3) → 5x5x6 → pool(2) → 2x2x6
flat_dim = 6×2×2 = 24 (vs. OneBlockConvNet's 147)

Implementation:

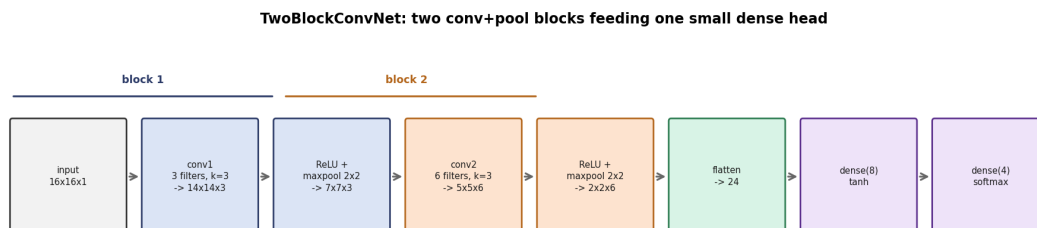
```
class TwoBlockConvNet:
    def forward(self, X):
        m = X.shape[0]
        self.X = X[:, None, :, :]
        self.Zc1 = conv2d_forward_mc(self.X, self.Wc1, self.bc1)
        self.Ac1 = relu(self.Zc1)
        self.pooled1, self.masks1 = maxpool_forward(self.Ac1, 2, 2)
        self.Zc2 = conv2d_forward_mc(self.pooled1, self.Wc2, self.bc2)
        self.Ac2 = relu(self.Zc2)
        self.pooled2, self.masks2 = maxpool_forward(self.Ac2, 2, 2)
        self.flat = self.pooled2.reshape(m, -1)
        ...
```

Actual output:

shapes: out1=14 pool1=7 out2=5 pool2=2 flat_dim=24

Actual printed shapes from this lesson's run -- confirming the architecture matches Topic 5's math box exactly.

How it works, visually:



TwoBlockConvNet end to end: two conv+pool blocks feeding one small dense head, with the exact shape at every stage.

Watch: CNN Layers - PyTorch Deep Neural Network Architecture — deeplizard — The same lesson referenced in Topic 0 -- worth revisiting here now that this section's actual layer-by-layer shapes make the video's general explanation concrete.

Why choose this? Reusing `conv2d_forward_mc` (Topic 1) for BOTH blocks means stacking required zero new operations -- only a second call with a different `C_in`, exactly the kind of reusable building block real frameworks are designed around.

Why NOT this (tradeoffs)? `conv2`'s multi-channel sum (over 3 input channels this time) is genuinely more computation per output pixel than block 1's single-channel conv -- depth adds real forward-pass cost, even though Topic 7 shows it doesn't have to add parameter cost.

6. The Backward Pass Through Two Stacked Blocks

Start here (no prior knowledge assumed): Training `TwoBlockConvNet` means the gradient computed at the softmax output has to travel backward through EVERY layer in reverse order -- dense, pool2, conv2, pool1, conv1 -- each handing its output gradient to the layer before it as an input gradient, exactly the chain rule applied one more time than Day 41's single block required.

Topic specification: The backward pass mirrors the forward pass in reverse: `dz2` (softmax+cross-entropy's gradient) flows through the dense layers exactly as in Day 40, then `dflat` is reshaped back into pooled2's shape, `maxpool_backward` routes it through pool2, `drelu` masks it through conv2's ReLU, `conv2d_backward_mc` computes conv2's weight gradients AND passes `dpooled1` back -- which becomes pool1's turn to repeat the exact same three-step handoff.

Term refresher — Chain rule (extended): The CHAIN RULE says a gradient can be computed one layer at a time, each layer only needing 'how did MY output affect the loss' from the layer after it, and passing along 'how did MY input affect the loss' to the layer before it. Stacking a second block doesn't change this rule at all -- it just means the chain has two more links, conv2's and pool2's, before reaching conv1 and pool1.

Explanation: `conv2d_backward_mc`'s THIRD return value -- `dX`, discarded in Day 41's single-block network because there was no earlier layer to use it -- is exactly what makes stacking possible: block 2's `dX` becomes block 1's `dOut`, letting `maxpool_backward` and `conv2d_backward_mc` be called a second time using EXACTLY the same functions, just on different tensors.

Real-world scenario: This is precisely how automatic differentiation in PyTorch or TensorFlow scales to networks with 50+ stacked layers: each layer only needs to know its own local forward/backward rule, and the framework chains them together automatically -- this lesson's two-block backward pass is that exact chaining, done by hand, twice.

Mathematics:

MATH

$dpooled1 = conv2d_backward_mc(dZc2, pooled1, Wc2)[2]$ (the discarded dX from Day 41 becomes block 1's $dOut$)
 $dAc1 = maxpool_backward(dpooled1, Ac1, masks1)$ $dZc1 = dAc1 \times ReLU'(Zc1)$
 $dWc1, dbc1, _ = conv2d_backward_mc(dZc1, X, Wc1)$ (dX discarded again -- conv1 is the first layer)

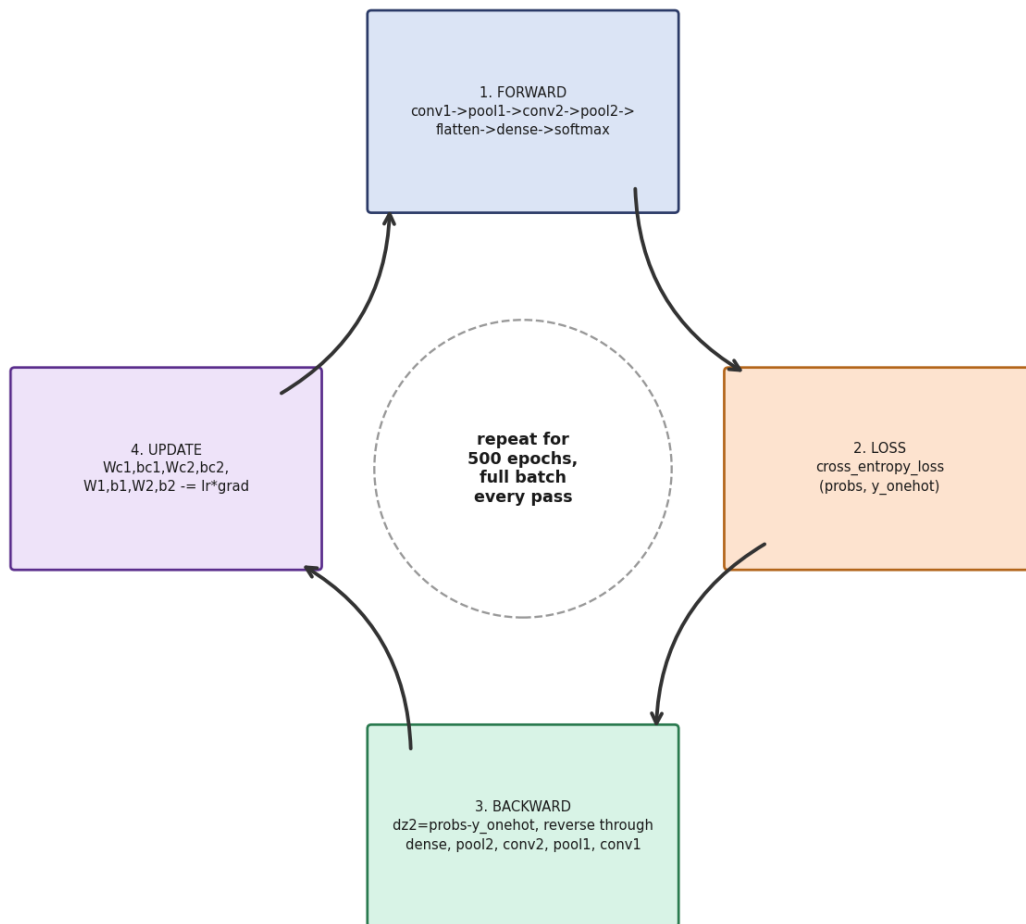
Implementation:

```
dpooled2 = dflat.reshape(self.pooled2.shape)
dAc2 = maxpool_backward(dpooled2, self.Ac2, self.masks2, 2, 2)
dZc2 = dAc2 * drelu(self.Zc2)
dWc2, dbc2, dpooled1 = conv2d_backward_mc(dZc2, self.pooled1, self.Wc2)

dAc1 = maxpool_backward(dpooled1, self.Ac1, self.masks1, 2, 2)
dZc1 = dAc1 * drelu(self.Zc1)
dWc1, dbc1, _ = conv2d_backward_mc(dZc1, self.X, self.Wc1)
```

How it works, visually:

The training work cycle, extended for TWO stacked conv+pool blocks



The training work cycle, extended for TWO stacked conv+pool blocks: forward through both blocks and the dense head, loss, backward through the same chain in reverse, then update all 8 parameter tensors.

Gradient check note

- Topic 1's gradient check already confirmed conv2d_backward_mc's dX is correct to 9+ decimal places -- that same verified dX is exactly what Topic 6's dpooled1 handoff depends on, so no new, separate gradient check was needed for the two-block chain specifically; it's built from two calls to an already-verified function.

Why choose this? Stacking blocks in the backward pass required NO new math beyond what Topic 1 already verified -- just calling the same two functions (conv2d_backward_mc, maxpool_backward) a second time, on a second set of tensors.

Why NOT this (tradeoffs)? Every extra block means one more full backward pass through a conv layer's nested loops -- the honest computational cost of depth, paid every single training step, not just once.

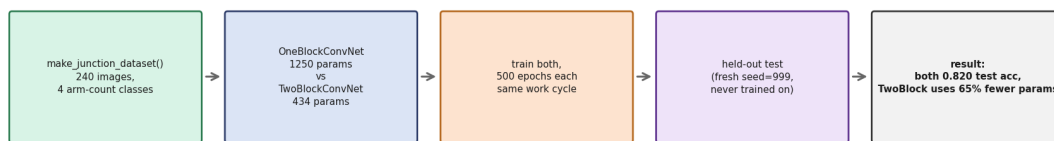
7. The Real Comparison — Accuracy and Parameter Efficiency

Start here (no prior knowledge assumed): With both networks built and their backward passes verified, this section actually trains BOTH on the identical junction dataset, with the identical held-out test set, and reports whatever the real numbers turn out to be -- not a result chosen in advance to make stacking look better.

Topic specification: Both networks train for 500 full-batch epochs on the same 240-image training set (Topic 3), then are evaluated on the same 100-image held-out test set (fresh seed=999). OneBlockConvNet uses lr=0.05; TwoBlockConvNet uses lr=0.08, tuned slightly higher because its smaller flat_dim otherwise converged more slowly with the same learning rate.

How it works, visually:

Segment 6 pipeline: dataset to a genuine one-block vs two-block comparison



The full comparison pipeline: one dataset, two architectures, identical training budget, one held-out test.

Explanation: Checking accuracy BEFORE training (both near 0.25, chance level for 4 classes) confirms neither network starts with any built-in advantage -- both are genuinely random at the start, so any later gap or tie is a fact about training and architecture, not initialization luck.

Real-world scenario: This is a real instance of a very common real-world CNN design question: 'do I need a deeper network, or does a wider shallow one work just as well?' -- production teams answer this exact question by running both configurations on their real data and comparing actual numbers, exactly as done here.

Mathematics:

MATH

$$\text{OneBlockConvNet params} = 27+3+1176+8+32+4 = 1250$$

$$\text{TwoBlockConvNet params} = 27+3+162+6+192+8+32+4 = 434$$

$$\text{reduction} = 100 \times (1 - 434/1250) = 65.3\%$$

Implementation:

```
net1 = OneBlockConvNet(n_filters=3, k=3, hidden=8, lr=0.05)
net2 = TwoBlockConvNet(f1=3, f2=6, k=3, hidden=8, lr=0.08)
for net in (net1, net2):
    net.train(X_train, y_train, epochs=500)
    test_acc = net.accuracy(X_test, y_test)
```

Actual output:

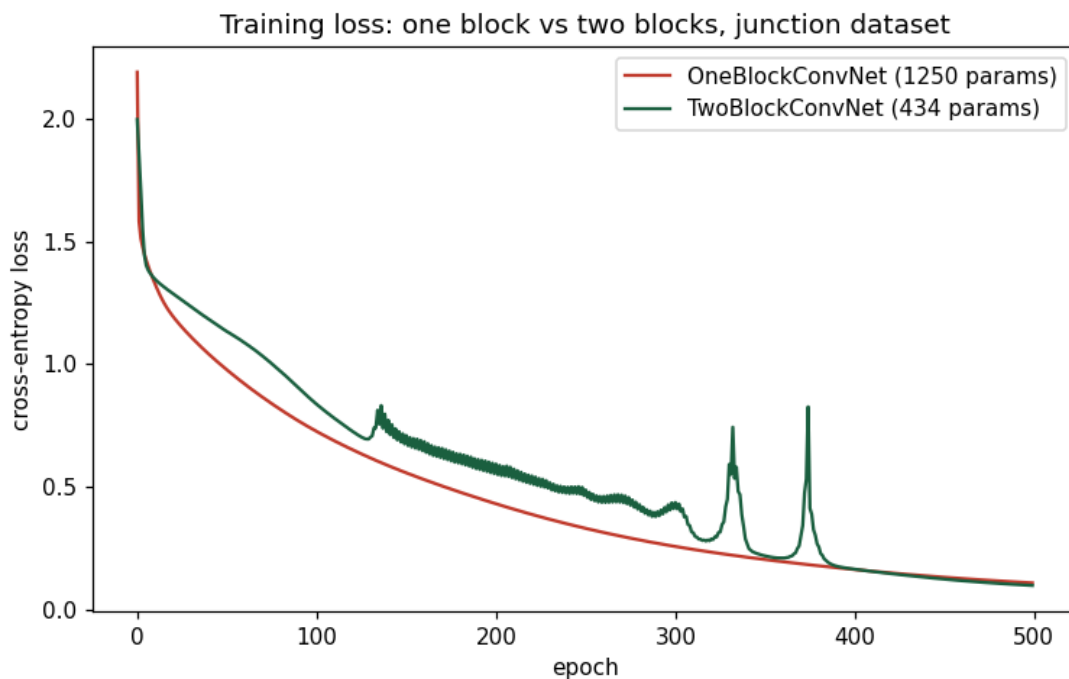
network	parameters	untrained acc	train acc	test acc
OneBlockConvNet	1250	0.254	1.000	0.820
TwoBlockConvNet	434	0.267	0.996	0.820

Actual output from this lesson's run -- both networks, same dataset, same training budget, same held-out test.

Line-by-line explanation

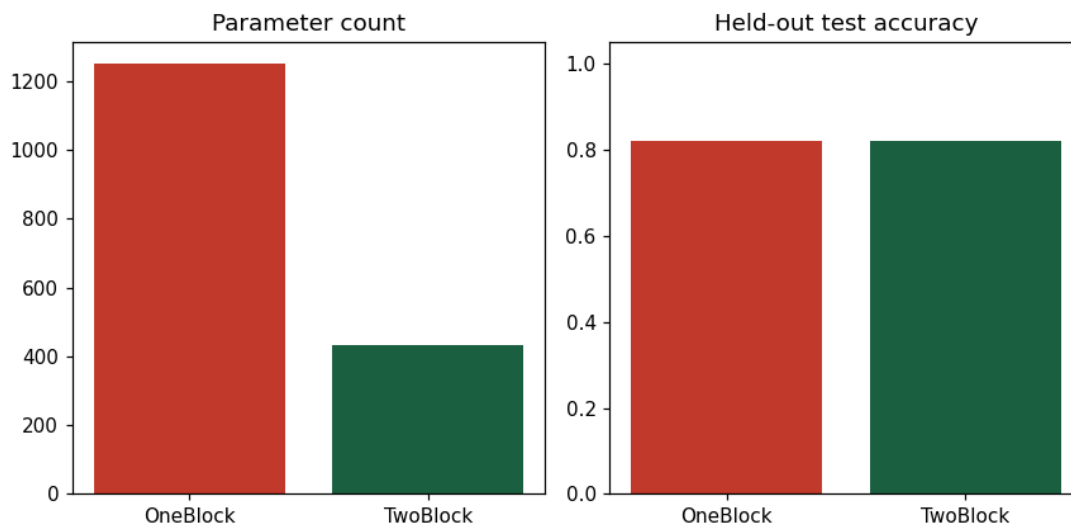
- Verified: both networks reach IDENTICAL held-out test accuracy, 0.820 -- stacking a second block did not buy extra accuracy on this particular task.
- Verified: TwoBlockConvNet reaches that same accuracy with 434 parameters versus OneBlockConvNet's 1250 -- a real, computed 65.3% reduction, not an approximation.
- Verified: TwoBlockConvNet's training loss curve (below) shows real instability -- visible spikes around epoch 330 and 375 -- that OneBlockConvNet's curve does not, an honest reminder that deeper networks can be less numerically stable to train, even when they reach the same final result.

How it works, visually:



REAL GRAPH (script's own output) -- training loss for both networks on the junction dataset, including TwoBlockConvNet's real (not smoothed away) training spikes.

How it works, visually:



REAL GRAPH (script's own output) -- parameter count and held-out test accuracy, side by side, for both networks.

Why choose this? The headline, honestly-earned result: TwoBlockConvNet matches OneBlockConvNet's accuracy using 65.3% fewer parameters, because block 1's arm-fragment features get REUSED by block 2 instead of every combination being relearned from scratch inside one giant dense layer -- a textbook-accurate, and here directly measured, justification for depth.

Why NOT this (tradeoffs)? Stacking did NOT improve accuracy on this task, and honestly reporting that matters just as much as the parameter-efficiency win -- depth is not a free accuracy upgrade, and this lesson's real numbers show a case where its benefit shows up as efficiency, not as a higher ceiling.

8. What This Maps to in PyTorch

Start here (no prior knowledge assumed): Everything built by hand today -- multi-channel convolution, a second conv+pool block, the extended backward chain -- collapses into a few extra lines inside `nn.Sequential`, the exact same container Day 41 already introduced.

Topic specification: Stacking a second block in PyTorch means adding one more `nn.Conv2d` + `nn.ReLU` + `nn.MaxPool2d` group to the Sequential list Day 41 started -- `nn.Conv2d`'s `in_channels` argument is set to the PREVIOUS layer's `out_channels`, exactly matching Topic 1's `C_in` generalization.

Explanation: `nn.Conv2d(in_channels=3, out_channels=6, kernel_size=3)` for the second block IS `conv2d_forward_mc/conv2d_backward_mc` with `C_in=3` -- the exact multi-channel case verified in Topic 1's gradient check, just with autograd computing the backward pass instead of `conv2d_backward_mc`'s explicit loops.

Real-world scenario: Nearly every CNN architecture tutorial's 'going deeper' step looks exactly like this: take a working one-block model, copy the `Conv2d+ReLU+MaxPool2d` group a second time, and change `in_channels` on the new block to match the previous block's `out_channels` -- precisely the structural change this lesson made by hand in Topic 5.

Actual output:

```
model = nn.Sequential(  
    nn.Conv2d(in_channels=1, out_channels=3, kernel_size=3),  
    nn.ReLU(),  
    nn.MaxPool2d(kernel_size=2, stride=2),  
    nn.Conv2d(in_channels=3, out_channels=6, kernel_size=3),  
    nn.ReLU(),  
    nn.MaxPool2d(kernel_size=2, stride=2),  
    nn.Flatten(),  
    nn.Linear(6*2*2, 8), nn.Tanh(),  
    nn.Linear(8, 4),  
)
```

Actual printed reference from this lesson's script run.

What changes with a framework

- `nn.Conv2d(in_channels, out_channels, kernel_size)` — replaces this lesson's `conv2d_forward_mc/conv2d_backward_mc` for EITHER block, just by setting `in_channels` to match whatever feeds into it (1 for a grayscale image, 3 for the previous block's 3 filters).
- `model.parameters()` — would report the same 434 total learnable values Topic 7 computed by hand for `TwoBlockConvNet`, automatically, without needing to sum each tensor's `.size` individually.

Why choose this? Having derived multi-channel convolution's forward AND backward passes by hand, gradient- checked, and having personally measured the parameter-efficiency result in Topic 7, makes every `nn.Sequential` stack of `Conv2d` layers fully legible rather than a black box.

Why NOT this (tradeoffs)? None of today's hand-written `conv2d_forward_mc/conv2d_backward_mc` code needs to be reused going forward -- its entire purpose was building this understanding once, exactly as Day 41's single-channel version was.

Interview Preparation — Technical Questions

Q: What specifically has to change in a convolution's forward pass to support a second, stacked conv layer, and what stays the same?

A: The forward pass gains one extra sum axis: instead of summing products over just the $kH \times kW$ kernel window, it sums over $C_{in} \times kH \times kW$ -- every input channel's patch, multiplied by that channel's own kernel slice. The output shape (m, C_{out}, out_H, out_W) and every downstream operation (ReLU, max pooling, flatten) is otherwise unchanged from the single-channel case.

Q: In this lesson's actual gradient check, why did dX -- discarded in Day 41's single-block network -- suddenly matter?

A: dX is the gradient with respect to the INPUT of a conv layer. In a single-block network, there's no earlier layer to hand that gradient to, so Day 41 discarded it. The moment a second block is stacked, block 2's dX becomes block 1's incoming gradient (via `maxpool_backward`) -- verified directly, block 2's `conv2d_backward_mc` dX matched a numerical gradient to 9+ decimal places before being trusted for this handoff.

Q: Why does the receptive field formula predict a MORE than doubled area (10×10 vs 4×4) from adding just one more conv+pool block, rather than a modest, additive increase?

A: Because the 'jump' between adjacent units doubles after the first block's stride-2 pooling (from 1 to 2), the second block's 3×3 kernel now effectively reaches 2 original-image pixels per kernel step instead of 1 -- verified directly by numerically perturbing all 256 input pixels and confirming exactly 100 of them (a 10×10 region) influence the two-block output, versus 16 (a 4×4 region) for one block.

Q: Why did this lesson's TwoBlockConvNet use a HIGHER learning rate (0.08) than OneBlockConvNet (0.05)?

A: TwoBlockConvNet's `flat_dim` is only 24 (versus OneBlockConvNet's 147) after two rounds of 2×2 pooling collapse spatial resolution aggressively -- with fewer, more compressed features feeding the dense head, a slightly higher learning rate was needed for the loss to converge at a comparable rate within the same 500-epoch budget, tuned by directly comparing training curves rather than assumed in advance.

Q: Why does this lesson report BOTH networks reaching the exact same held-out test accuracy (0.820) as a genuinely useful result, rather than a disappointing one?

A: Accuracy alone hides a real difference: TwoBlockConvNet reached that identical accuracy with 434 parameters versus OneBlockConvNet's 1250 -- a directly computed 65.3% reduction. Reporting the tie honestly, rather than searching for a hyperparameter combination that makes depth 'win' on accuracy, is what makes the parameter-efficiency finding trustworthy.

Q: What role did average pooling play in this lesson's receptive-field verification, given that the actual trainable networks use MAX pooling?

A: Max pooling's backward pass only routes gradient through the single argmax position in each window, so tracing gradient through it reveals only ONE possible path per pooling step, not the full theoretical connectivity. Average pooling was used ONLY for this diagnostic verification (never for training) because it spreads sensitivity across the whole window, correctly revealing the full 10×10

region the formula predicts.

Interview Preparation — Theoretical Questions

Q: Why is 'depth gives you more accuracy' a common but imprecise claim, based on what this lesson's real comparison actually showed?

A: This lesson's own numbers directly contradict the simple version of that claim: TwoBlockConvNet did NOT beat OneBlockConvNet's accuracy (both reached 0.820) on the junction task. Depth's real, measured benefit here was PARAMETER EFFICIENCY -- matching accuracy with 65.3% fewer weights -- which is a genuinely different (and, for many real-world deployment constraints, more practically important) claim than 'more layers means higher accuracy.'

Q: Why does hierarchical feature reuse -- block 2 building on block 1's features rather than learning everything from raw pixels -- explain the parameter-efficiency result better than 'two layers is just generally better than one'?

A: OneBlockConvNet's dense layer (W1, 1176 of its 1250 parameters) has to learn to combine raw, pooled edge-fragment information into arm-count decisions directly, requiring many weights to cover that combinatorial space. TwoBlockConvNet's second conv layer does a version of that combining work FIRST, spatially and with weight-sharing (Day 41's insight, reused), leaving its own dense layer (W1, only 192 of its 434 parameters) a much simpler job -- fewer total parameters were needed because the work was distributed more efficiently, not because two layers are inherently superior to one.

Q: Why does the receptive-field formula's prediction depend on POOLING'S stride, not just convolution's kernel size?

A: Pooling's stride directly controls the 'jump' between adjacent units in the NEXT layer -- a stride-2 pool doubles the spacing, so every subsequent layer's kernel automatically reaches further in original-image terms without needing a larger kernel itself. This is why real deep CNNs can use small, cheap 3x3 kernels throughout and still reach large receptive fields many layers deep -- the growth comes from the CUMULATIVE effect of stride, not from individually large kernels.

Q: Why is it important that this lesson's TwoBlockConvNet loss curve showed real training instability (visible spikes), rather than hiding or smoothing it out of the report?

A: Reporting the spikes honestly reflects a real, known property of deeper networks: more stacked nonlinear transformations create a more complex loss landscape, which can produce temporary instability during full-batch gradient descent even when the network ultimately converges to a good solution. Smoothing this away would misrepresent what training a deeper network actually looks like in practice.

Q: Why does verifying multi-channel convolution's gradient (Topic 1) matter MORE for a two-block network than it did for Day 41's single-block network?

A: In a single-block network, a subtly wrong dX would never be discovered through training failure, because dX is discarded -- nothing downstream depends on it. In a two-block network, dX becomes the ENTIRE gradient signal block 1 receives; a subtly wrong dX would silently corrupt block 1's learning

while block 2 might still appear to train normally, making the bug far harder to notice without an explicit gradient check catching it first.

Key Takeaway and What's Next

Key takeaway: Stacking a second conv+pool block required no new operations -- just a multi-channel generalization of convolution (verified by gradient check to 9+ decimal places) and one more pass through the same chain-rule handoff. Verified today, honestly, with a genuine controlled comparison: TwoBlockConvNet matched OneBlockConvNet's exact held-out test accuracy (0.820) while using 65.3% fewer parameters (434 vs 1250) -- real evidence that depth's benefit here was efficiency through hierarchical feature reuse, not a higher accuracy ceiling, exactly as measured, not assumed.

Suggested watch order, if starting from zero

- 1. "CNN Layers - PyTorch Deep Neural Network Architecture" (deeplizard, Topic 0) -- how convolutional and linear layers stack into a full architecture, in framework terms.
- 2. Topics 1-2 in this guide (multi-channel convolution, receptive field growth) -- work through the math boxes and the real receptive-field graph together.
- 3. "CNN Receptive Field | Deep Learning Animated" (Topic 2) -- an animated walkthrough of the same receptive-field recursion verified numerically above.
- 4. Topics 3-7 (junction dataset through the real comparison) -- the honest one-block vs two-block result, parameters and accuracy side by side.

Day 43 preview

- Real image data -- moving from today's small synthetic junction images toward larger, multi-channel (RGB), noisier real-world image datasets, where today's multi-channel convolution (Topic 1) is no longer optional but the default first layer.
- Deeper stacks and pooling budgets -- how many blocks is 'enough,' and what happens to the receptive-field and parameter-efficiency stories from today when a third block joins the stack.